МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

Выполнил:
Е.И. Мицкевич
Студент группы ПО-13
Проверил:
А.Д. Кулик
преп.-стажёр кафедры ИИТ

Брест 2026

Цель работы: освоить приемы тестирования кода на примере использования пакета pytest.

Ход работы

Задание 1: Написание тестов для мини-библиотеки покупок (shopping.py)
1. Создайте файл test_cart.py. Реализуйте следующие тесты:
• Проверка добавления товара: после add_item("Apple", 10.0) в корзине должен быть один элемент.
• Проверка выброса ошибки при отрицательной цене.
• Проверка вычисления общей стоимости (total()).
2. Протестируйте метод apply_discount с разными значениями скидки:
• 0% - цена остаётся прежней
• 50% - цена уменьшается вдвое
• 100% - цена становится ноль
• < 0% и > 100% - должно выбрасываться исключение
Используйте @pytest.mark.parametrize
3. Создайте фикстуру empty_cart, которая возвращает пустой экземпляр Cart
@pytest.fixture
def empty_cart():
return Cart()
Используйте эту фикстуру в тестах, где нужно создать новую корзину.
4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:
import requests
def log_purchase(item):
requests.post("https://example.com/log", json=item)
• Замокайте requests.post, чтобы не было реального HTTP-запроса
• Убедитесь, что он вызывается с корректными данными
5. Добавьте поддержку купонов:
def apply_coupon(cart, coupon_code):
coupons = {"SAVE10": 10, "HALF": 50}
if coupon_code in coupons:
cart.apply_discount(coupons[coupon_code])
else:
raise ValueError("Invalid coupon")
• Напишите тесты на apply_coupon
• Замокайте словарь coupons с помощью monkeypatch или patch.dict

```
importpytest
fromunittest.mock importMock


# 1.1 ПРОВЕРКА ДОБАВЛЕНИЯ ТОВАРА
deftest_add_item():

    cart =Cart()
    cart.add_item("Apple", 10.0)

    # Assert (Проверка)
    assertcart.get_item_count() ==1
    assertlen(cart.items) ==1
    assertcart.items[0]["name"] =="Apple"
```

```
    assertcart.items[0]["price"] ==10.0

# 1.2 ПРОВЕРКА ВЫБРОСА ОШИБКИ ПРИ ОТРИЦАТЕЛЬНОЙ ЦЕНЕ
deftest_negative_price():


    cart =Cart()

    withpytest.raises(ValueError) asexc_info:
        cart.add_item("Bad Item", -5.0)

    # Проверяем, что было выброшено именно ValueError
    assertexc_info.type ==ValueError

    # Проверяем сообщение об ошибке
    assert"Цена не может быть отрицательной"instr(exc_info.value)
    # Убеждаемся, что товар не был добавлен в корзину
    assertcart.get_item_count() ==0


# 1.3 ПРОВЕРКА ВЫЧИСЛЕНИЯ ОБЩЕЙ СТОИМОСТИ


deftest_total():


    cart =Cart()
    cart.add_item("Apple", 10.0)
    cart.add_item("Banana", 15.5)
    cart.add_item("Orange", 8.75)

    expected_total =10.0+15.5+8.75 # = 34.25
    assertcart.total() ==expected_total


# 2 ПРОВЕРКА ВВЫЧИСЛЕНИЯ ЦЕНЫ ПО СКИДКЕ


@pytest.mark.parametrize("sale, cost, result", [
    (0, 30, 30),
    (50, 20, 10),
    (100, 50, 0),
])


deftest_apply_discount(sale, cost, result):
    assert(sale, cost) ==result


@pytest.mark.parametrize("discount", [
    -1,    # Отрицательная скидка
    -50,   # Отрицательная скидка
    101,   # Скидка > 100%
    150   # Скидка > 100%
])


deftest_apply_discount_invalid(discount):

    withpytest.raises(ValueError, match="Процент скидки должен быть в диапазоне от 0 до
100"):
        empty_cart.apply_discount(discount)


# 3. С ФИКСТУРОЙ
```

```python
@pytest.fixture
defempty_cart():

    returnCart()

deftest_add_single_item(empty_cart):

    #Используем фикстуру empty_cart вместо создания корзины вручную.
    empty_cart.add_item("Apple", 10.0)

    assertempty_cart.get_item_count() ==1
    assertlen(empty_cart.items) ==1
    assertempty_cart.items[0]["name"] =="Apple"
    assertempty_cart.items[0]["price"] ==10.0


# 4. МОКИРОВАНИЕ


deftest_log_purchase_calls_requests_post(mocker):
    """
    Тест проверяет, что log_purchase вызывает requests.post
    с правильными параметрами
    """
    # 1. Мокаем requests.post
    mock_post =mocker.patch('requests.post')

    # 2. Данные для логирования
    item ={"name": "Apple", "price": 10.0}

    # 3. Вызываем тестируемую функцию
    log_purchase(item)

    # 4. Проверяем, что requests.post был вызван ровно 1 раз
    mock_post.assert_called_once()

    # 5. Проверяем, что вызван с правильными аргументами
    mock_post.assert_called_with(
        "https://example.com/log",
        json=item
    )


deftest_log_purchase_called_with_correct_data(mocker):
    """
    Тест проверяет, что в requests.post передаются корректные данные
    """
    # Мокаем requests.post
    mock_post =mocker.patch('requests.post')

    # Тестовые данные
    test_item ={"name": "Banana", "price": 15.5}

    # Вызываем функцию
    log_purchase(test_item)

    # Получаем аргументы, с которыми был вызван мок
    args, kwargs =mock_post.call_args

    # Проверяем URL
    assertargs[0] =="https://example.com/log"

    # Проверяем, что переданные данные совпадают
```

```
    assertkwargs["json"] ==test_item
    assertkwargs["json"]["name"] =="Bababababanana"
    assertkwargs["json"]["price"] ==15.5


#5. ЧЕРЕЗ ОБЕЗЬЯНКУ


deftest_apply_coupon_save10(monkeypatch):
    """
    Тест проверяет применение купона SAVE10 (10% скидка)
    Используем monkeypatch для мокирования словаря coupons
    """
    # Arrange - Создаём корзину с товаром
    cart =Cart()
    cart.add_item("Apple", 100.0)

    # Мокаем словарь coupons через monkeypatch
    test_coupons ={"SAVE10": 10}
    monkeypatch.setattr('shopping.apply_coupon.__code__.co_consts', test_coupons)
    # Более простой способ: мокаем через глобальную переменную
    importshopping
    monkeypatch.setattr(shopping, 'coupons', test_coupons)

    apply_coupon(cart, "SAVE10")

    assertcart.total() ==90.0 # 100 - 10% = 90
    assertcart.items[0]["price"] ==90.0


deftest_apply_coupon_half(monkeypatch):
    """
    Тест проверяет применение купона HALF (50% скидка)
    """
    # Arrange
    cart =Cart()
    cart.add_item("Laptop", 1000.0)

    # Мокаем словарь coupons
    test_coupons ={"HALF": 50}
    importshopping
    monkeypatch.setattr(shopping, 'coupons', test_coupons)


    apply_coupon(cart, "HALF")

    assertcart.total() ==500.0 # 1000 - 50% = 500
    assertcart.items[0]["price"] ==500.0
```

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случае, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

```
importpytest


defbubbleSort(arr):
    sizeArr=len(arr)
    foriinrange(sizeArr):
        forjinrange(0, sizeArr-i-1):
```

```
            ifarr[j] <arr[j+1]:
                temp=arr[j]
                arr[j] =arr[j+1]
                arr[j+1] =temp
    returnarr


@pytest.mark.parametrize("arr, result_arr", [
    ([1,2,3,4,5], [5,4,3,2,1]),
    ([5,4,3,2,1], [5,4,3,2,1]),
    ([1,1,1,1,1], [1,1,1,1,1]),
    ([-1,-2,-3,-4,0], [0,-1,-2,-3,-4] ),
    ([5,5,5,5,5], [5,5,5,5,5]),
    ([] ,[]),
    ([2] ,[2]),
    ([1, 2], [2, 1]),
    ([-5, 5], [5, -5])
])


deftest_bubbleSort(arr, result_arr):
    result_test=bubbleSort(arr)
    assertresult_test==result_arr


# ИСКЛЮЧИТЕЛЬНЫЕ


deftest_bubbleSort_with_none():
    """Тест: передача None вместо массива"""
    withpytest.raises(TypeError) asexc_info:
        bubbleSort(None)
    assert"object of type 'NoneType' has no len()"instr(exc_info.value)



deftest_bubbleSort_with_non_list():
    """Тест: передача не списка"""
    withpytest.raises(TypeError):
        bubbleSort(42)

    withpytest.raises(TypeError):
        bubbleSort("string")

    withpytest.raises(TypeError):
        bubbleSort(3.14)
```

```
D:\PYTHON\TESTS\SPP\6>py -m pytest test_1.py
============================================= test session starts ==
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\PYTHON\TESTS\SPP\6
plugins: anyio-4.12.1, mock-3.15.1
collected 11 items

test_1.py ...........

============================================== 11 passed in 0.05s ===
```

```
importpytest


deffactorial(number):


    # Добавляем проверку на отрицательные числа
    ifnumber<0:
        raiseValueError("Факториал определён только для неотрицательных чисел")
```

```
    resultFactor=1
    ifnumber==0:
        returnresultFactor
    foriinrange(1, number+1):
        resultFactor*=i
    returnresultFactor



defcountVar(value):


    ifnotisinstance(value, int):
        raiseTypeError("Аргумент должен быть целым числом")

    ifvalue<0:
        raiseValueError("Аргумент не может быть отрицательным")

    valueOfBigStep=value//2
    valueOfLittleStep=value%2
    result=0

    whilevalueOfBigStep>=0:
        result+=factorial(valueOfLittleStep+valueOfBigStep) //(
            factorial(valueOfLittleStep) *factorial(valueOfBigStep)
        )
        valueOfBigStep-=1
        valueOfLittleStep+=2

    returnresult


# ТЕСТЫ ДЛЯ ФАКТОРИАЛА


@pytest.mark.parametrize("number, expected", [
    (0, 1),
    (1, 1),
    (2, 2),
    (5, 120),
    (10, 3628800)
])
deftest_factorial(number, expected):
    """Тестирование факториала"""
    result_test=factorial(number)
    assertresult_test==expected



deftest_factorial_negative():
    """Тест: факториал отрицательного числа"""
    withpytest.raises(ValueError, match="Факториал определён только для неотрицательных
чисел"):
        factorial(-1)

    withpytest.raises(ValueError):
        factorial(-5)



# ТЕСТЫ ДЛЯ COUNTVAR


@pytest.mark.parametrize("number, expected", [
    (0, 1),
```

```
    (1, 1),
    (2, 2),
    (3, 3),
    (4, 5),
    (5, 8),
    (10, 89)
])
deftest_countVar(number, expected):
    """Тестирование countVar"""
    result_test=countVar(number)
    assertresult_test==expected



deftest_countVar_negative():
    """Тест: отрицательное значение"""
    withpytest.raises(ValueError, match="Аргумент не может быть отрицательным"):
        countVar(-1)

    withpytest.raises(ValueError):
        countVar(-5)



deftest_countVar_non_integer():
    """Тест: передача не целого числа"""
    withpytest.raises(TypeError, match="Аргумент должен быть целым числом"):
        countVar(3.5)

    withpytest.raises(TypeError):
        countVar("5")

    withpytest.raises(TypeError):
        countVar([5])
```

```
D:\PYTHON\TESTS\SPP\6>py -m pytest test_2.py
============================================= test session starts =
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\PYTHON\TESTS\SPP\6
plugins: anyio-4.12.1, mock-3.15.1
collected 15 items

test_2.py ...............

============================================== 15 passed in 0.07s ==
```

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

5) Реализуйте и протестируйте метод int indexOfDifference(String str1, String str2), который сравнивает две строки и возвращает индекс той позиции, в которой они различаются. Например, indexOfDifference("i am a machine" , "i am a robot") должно вернуть 7.

Спецификация метода:

indexOfDifference (None , None) = TypeError
indexOfDifference ("", "") = -1
indexOfDifference ("", "abc ") = 0
indexOfDifference ("abc ", "") = 0
indexOfDifference ("abc ", " abc ") = -1
indexOfDifference ("ab", " abxyz ") = 2
indexOfDifference (" abcde ", " abxyz ") = 2
indexOfDifference (" abcde ", "xyz") = 0

Решение

```
importpytest


defindexOfDifference(str1, str2):
    # Проверка на None
    ifstr1isNoneorstr2isNone:
        raiseValueError("Нельзя использовать none значения")

    # Если обе строки пустые
    ifstr1==""andstr2==="":
        return-1

    # Если одна из строк пустая, а другая нет
    ifstr1==""orstr2==="":
        return0
    i=0
    while(i<len(str1) andi<len(str2)):
        ifstr1[i] ==str2[i]: returni

    return-1



# ТЕСТЫ


@pytest.mark.parametrize("str_1, str_2, result", [
    ("", "", -1),
    ("", "abc", 0),
    ("abc", "", 0),
    ("abc", "abc", -1),
    ("ab", "abxyz", 2),
    ("abcde", "abxyz", 2),
    ("abcde", "xyz", 0),
    ("qwertyuiopasdfghjkl", "qwertyuiopasdfghjkl", -1)
])


deftest_indexOfDifference(str_1, str_2, result):


    result_test=indexOfDifference(str_1, str_2)
    assertresult_test==result



deftest_indexOfDifference_none():


    withpytest.raises(ValueError, match="Нельзя использовать none значения"):
        indexOfDifference(None, None)
```

```
D:\PYTHON\TESTS\SPP\6>py -m pytest test_1.py
================================================= test session starts ===
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\PYTHON\TESTS\SPP\6
plugins: anyio-4.12.1, mock-3.15.1
collected 11 items

test_1.py ...........

================================================== 11 passed in 0.05s ===
```

Вывод: были освоены приемы тестирования кода на примере использования пакета pytest.